High-Performance Linear Algebra Kernels

ACF, Team 11

April 05, 2026

---

1. Test Environment

All benchmarks were collected on a single machine to ensure reproducibility.

Component	**Detail**
CPU	Apple M4 Pro (10P + 4E cores)
RAM	48 GB unified
Cache line	128 bytes
P-core L1d / L2	128 KB / 16 MB
E-core L1d / L2	64 KB / 4 MB
Compiler	LLVM clang 22.1.2 (Homebrew)
Standard	C++23, `-Wall -Wextra`
OS	macOS 26.3.1 (Darwin 25.3.0)

Table 1: Test environment

2. Correctness Testing

Before measuring performance, we verify correctness with a two-tier test suite compiled into a single binary (`tests/correctness.cpp`).

2.1. Hardcoded Tests

Ten deterministic tests with hand-computed expected values cover basic operation and edge cases:

Test	**What it covers**
`test_multiply_mv_row_major`	2×2 row-major MV
`test_multiply_mv_col_major`	2×2 col-major MV
`test_multiply_mm_naive`	2×2 naive MM
`test_multiply_mm_transposed_b`	2×2 transposed-B MM
`test_multiply_mm_tiled`	2×2 tiled MM
`test_mv_1x1`	1×1 scalar MV (both layouts)
`test_mm_1x1`	1×1 scalar MM (all three variants)
`test_mm_identity`	3×3 identity: $A \cdot I = A$
`test_mv_non_square`	2×3 rectangular MV (both layouts)
`test_mm_non_square`	2×3 × 3×2 rectangular MM (all three variants)

Table 2: Hardcoded correctness tests

2.2. Randomized Cross-Validation

Rather than hand-computing expected outputs for large matrices, we exploit the invariant that different kernel implementations must agree on the same mathematical operation. A seeded PRNG (`std::mt19937`, seed 42) generates random matrices with values in $[-100, 100]$, ensuring deterministic reproducibility.

Floating-point tolerance scales with the inner product length k as $k \times 10^{-9}$, accounting for accumulation error that grows with the number of additions.

Suite	**Trials**	**What it cross-validates**
`fuzz_mv`	1100	Row-major vs col-major MV across 22 dimension configs (square + non-square), 50 trials each
`fuzz_mm`	1300	Naive vs transposed-B vs tiled MM across 27 dimension configs including tile-boundary stress sizes (31, 33, 63, 65, 95, 97), 50 trials each
`fuzz_mm_tiles`	600	Tiled MM with 10 tile sizes (1, 2, 7, ..., 64) across 12 matrix sizes, 5 trials each, against naive as reference

Table 3: Randomized cross-validation suites (3000 total trials)

Tile-boundary sizes (e.g. 31, 33 around the default tile of 32) specifically target off-by-one bugs in the tiling remainder logic. Non-square dimensions (e.g. $3 \times 4 \times 2$) catch errors that only manifest when $m \neq k \neq n$.

3. Benchmarking Results

All timings measured with `std::chrono::high_resolution_clock`. Matrix-vector runs use 100 iterations; matrix-matrix runs use 100 iterations for $n \leq 256$ and 10 iterations for $n \geq 512$ to keep total runtime practical.

3.1. Compiler Optimisation Levels

Table 4 shows mean latency across all four compiler optimisation levels. The `best` column indicates which level produced the lowest mean.

Kernel	n	O0 (ms)	O1 (ms)	O2 (ms)	O3 (ms)	Best
`mv_row_major`	64	0.0044	0.0015	0.0007	0.0008	O2
`mv_col_major`	64	0.0050	0.0011	0.0004	0.0004	O2
`mm_naive`	64	0.319	0.095	0.096	0.096	O1
`mm_transposed_b`	64	0.319	0.094	0.049	0.050	O2
`mm_tiled`	64	0.317	0.073	0.026	0.027	O2
`mv_row_major`	256	0.071	0.036	0.020	0.022	O2
`mv_col_major`	256	0.081	0.019	0.008	0.008	O2
`mm_naive`	256	45.6	10.5	10.6	10.6	O1
`mm_transposed_b`	256	20.6	9.0	5.3	5.3	O2
`mm_tiled`	256	20.4	4.6	2.3	2.3	O2
`mv_row_major`	512	0.278	0.179	0.102	0.101	O3
`mv_col_major`	512	0.320	0.070	0.033	0.033	O3
`mm_naive`	512	402	111	111	111	O1
`mm_transposed_b`	512	164	92	51	52	O2
`mm_tiled`	512	168	39	21	21	O2
`mv_row_major`	1024	1.10	0.84	0.48	0.52	O2
`mv_col_major`	1024	1.28	0.28	0.13	0.13	O2
`mm_naive`	1024	3405	934	937	926	O3
`mm_transposed_b`	1024	1304	796	494	491	O3
`mm_tiled`	1024	1340	322	193	191	O2

Table 4: Mean latency by optimisation level (aligned memory). Lower is better.

Key observations:

- **O0 → O1 is the largest single jump**, typically 3 to 4× for MM kernels. O1 enables inlining, dead-code elimination, and register allocation: the fundamentals.
- **O2 → O3 shows diminishing returns.** For `mm_naive` at $n = 1024$, O1 (934 ms) occasionally beats O2 (937 ms). Aggressive optimisations can increase code size and cause instruction-cache pressure.
- **Matrix-vector is memory-bound.** Even at O0 the MV kernels are sub-millisecond for $n \leq 512$; the bottleneck is DRAM bandwidth, not ALU throughput, so compiler optimisations have less room to help.

3.2. Kernel Comparison at O2

Table 5 isolates the effect of algorithm choice at a fixed optimisation level (O2), with standard deviation to gauge run-to-run variance.

Kernel	n	Mean (ms)	Stddev (ms)	vs naive
mm_naive	64	0.0938	0.0056	1.0×
mm_transposed_b	64	0.0477	0.0048	2.0×
mm_tiled	64	0.0256	0.0011	3.7×
mm_naive	256	10.58	0.21	1.0×
mm_transposed_b	256	5.27	0.20	2.0×
mm_tiled	256	2.30	0.04	4.6×
mm_naive	512	110.9	1.63	1.0×
mm_transposed_b	512	50.9	0.53	2.2×
mm_tiled	512	21.1	0.17	5.3×
mm_naive	1024	937	12.2	1.0×
mm_transposed_b	1024	494	5.4	1.9×
mm_tiled	1024	193	4.8	4.9×

Table 5: MM kernel comparison at `-O2` (aligned, 64-byte). Speedup relative to `mm_naive`.

The tiled kernel achieves a consistent 4 to 5× speedup over naive at large n. The transposed-B variant hovers around 2×, confirming that fixing B's access pattern alone captures half the available improvement.

4. Cache Locality Analysis

4.1. Matrix-Vector Multiplication: Row-Major vs Column-Major

In the **row-major** MV kernel, the inner loop iterates over consecutive columns:

```
for (int j = 0; j < cols; ++j)
    sum += matrix[i * cols + j] * vector[j];
```

Both `matrix[i*cols + j]` and `vector[j]` advance by stride 1, so every load exploits spatial locality within a cache line. On M4 Pro with 128-byte cache lines, each line delivers 16 contiguous doubles.

The **column-major** kernel reverses the loop order:

```
for (int j = 0; j < cols; ++j)
    for (int i = 0; i < rows; ++i)
        result[i] += matrix[j * rows + i] * vector[j];
```

The inner loop walks `matrix[j*rows + i]` with stride 1 through a single column (contiguous in column-major layout), while `vector[j]` is loop-invariant and hoisted. Both layouts therefore achieve good spatial locality for MV, which the benchmarks confirm: at O2, col-major (0.13 ms at $n = 1024$) is actually faster than row-major (0.48 ms) because the compiler hoists `vector[j]` and reduces the inner loop to a simple scaled vector accumulation.

4.2. Matrix-Matrix Multiplication: Naive vs Transposed B

In the naive i, j, k loop order:

```
sum += matrixA[i * colsA + k] * matrixB[k * colsB + j];
```

`matrixA[i*colsA + k]` has stride-1 access over k (good). But `matrixB[k*colsB + j]` has stride-`colsB` over k: each increment jumps an entire row, touching a new cache line every time. At $n = 1024$, the inner loop streams through $1024 \times 8 = 8$KB of non-contiguous B data, thrashing L1.

In the **transposed-B** variant, B is pre-transposed so that rows of B^T correspond to columns of B:

```
sum += matrixA[i * colsA + k] * matrixB_transposed[j * colsA + k];
```

Now both A and B^T have stride-1 access over k. The inner loop is a dot product of two contiguous rows, a pure streaming access pattern. This eliminates the column-stride penalty and explains the consistent 2× speedup.

4.3. Why Tiling Adds Another 2 to 3×

The naive kernel's working set for one full pass over k at fixed i, j is one row of A ($n \times 8$ bytes) plus one column of B ($n \times 8$ bytes). At $n = 1024$, that is 16 KB. It fits in L1, but each (i, j) pair loads a **different** column of B, so the total B traffic is $O(n^3)$ with no reuse.

Tiling restructures the loops so that a $T \times T$ block of A, a $T \times T$ block of B, and a $T \times T$ block of C are loaded once and reused T times. The working set per tile-triple is $3 \times T^2 \times 8$ bytes. At $T = 64$, this is 96 KB, or 75% of the M4 Pro P-core's 128 KB L1d, leaving room for loop state while maximising reuse before eviction.

5. Memory Alignment

We compare 64-byte aligned allocation (`std::aligned_alloc`) against default heap allocation (`new double[]`) across all kernels at `-O2`.

Kernel	n	Aligned (ms)	Unaligned (ms)	Δ
`mv_row_major`	1024	0.481	0.472	−2%
`mv_col_major`	1024	0.131	0.132	+1%
`mm_naive`	1024	937	954	+2%
`mm_transposed_b`	1024	494	495	+0.1%
`mm_tiled`	1024	193	192	−1%
`mm_naive`	64	0.094	0.120	+28%
`mm_transposed_b`	64	0.048	0.074	+54%
`mm_tiled`	64	0.026	0.034	+31%

Table 6: Aligned (64-byte) vs unaligned allocation at `-O2`.

At large n (≥ 256), alignment makes negligible difference. Deltas are within run-to-run variance ($\leq 2\%$). Apple M4 Pro handles unaligned loads in hardware with near-zero penalty when the access does not split a cache line, and at large n the dominant cost is cache misses from working-set pressure, not alignment.

At small n ($= 64$), alignment shows a measurable effect: up to 54% for `mm_transposed_b`. At this size the entire matrix fits in L1 ($64 \times 64 \times 8 = 32$ KB), so execution is fast enough that per-access overhead from crossing a cache-line boundary becomes visible. However, these are sub-0.1 ms operations where absolute differences are tens of microseconds.

The practical conclusion: 64-byte alignment is free insurance (the allocator overhead is negligible) and helps at small working sets, but it is not a meaningful optimisation lever for large matrices on modern Apple Silicon.

6. Inlining and Compiler Optimisations

6.1. Optimisation Level Impact

Table 4 shows the full picture. The key transitions:

O0 → O1 (3 to 4× for MM) O1 enables function inlining, register allocation, constant propagation, and dead-code elimination. For a kernel that is purely arithmetic and memory access, these are transformative: the unoptimised code pays for stack spills, redundant loads, and branch overhead on every iteration.

O1 → O2 (1 to 5× depending on kernel) O2 adds loop vectorisation and more aggressive LICM (loop-invariant code motion). `-Rpass=loop-vectorize` confirms that at O2, all five inner loops vectorise at width 2 with interleave count 4 (128-bit NEON, two doubles per register). This is the main reason `mm_tiled` drops from 322 ms (O1) to 193 ms (O2) at $n = 1024$.

O2 → O3 (≈1×) O3 adds aggressive unrolling, function specialisation, and vectorisation heuristic changes. In practice the effect is within measurement noise for our kernels. For `mm_naive` at $n = 1024$, O3 (926 ms) is marginally faster than O2 (937 ms), but the difference is not statistically significant given stddev of 12 ms.

6.2. Vectorisation Diagnostics

Compiling with `-Rpass=loop-vectorize` at O2 shows:

Kernel	Vectorised loop
`mv_row_major`	inner `j` loop: width 2, interleave 4
`mv_col_major`	inner `i` loop: width 2, interleave 4
`mm_naive`	inner `k` loop: width 2, interleave 4
`mm_transposed_b`	inner `k` loop: width 2, interleave 4
`mm_tiled`	inner `j` loop: width 2, interleave 4

Table 7: Loop vectorisation at `-O2` (confirmed via `-Rpass=loop-vectorize`)

Width 2 corresponds to 128-bit NEON registers holding two `double` values. Interleave count 4 means the compiler unrolls by 4 and uses 4 independent accumulators, hiding FMA latency. The `__restrict__` annotations on all pointer parameters are a prerequisite; without them, the compiler cannot prove that stores to `result` do not alias reads from `matrixA` or `matrixB`, blocking the vectoriser.

6.3. Drawbacks of Aggressive Optimisation

O3 occasionally **regresses** performance. At $n = 512$, `mm_naive` is 111 ms at both O1 and O2, but can spike at O3 due to over-aggressive unrolling increasing instruction-cache pressure. The M4 Pro P-core has 192 KB of L1i, which is large but not infinite. When the compiler fully unrolls and vectorises a triply-nested loop, the resulting code size can exceed what fits in the hot region of L1i, introducing instruction fetch stalls.

7. Profiling

7.1. Setup

Profiling was performed using the macOS Instruments toolchain:

- **Time Profiler**: `xcrun xctrace record --template 'Time Profiler'` for call-tree hot paths
- **CPU Counters**: `xcrun xctrace record --template 'CPU Counters'` for L1/L2 cache miss rates and branch mispredictions
- **LLVM PGO**: instrumented build (`-fprofile-instr-generate`) for per-function call counts
- **Compiler remarks**: `-Rpass` / `-Rpass-missed` for vectorisation diagnostics

A dedicated profiling driver (`src/profile_driver.cpp`) exercises each kernel at sizes where cache effects are prominent.

7.2. Findings

The profiling data confirmed what the benchmarks suggested:

1. **`mm_naive` inner loop is the bottleneck.** The Time Profiler shows 95%+ of execution time in the k-loop, with the majority of that time spent on loads from `matrixB[k*colsB + j]`, the stride-`colsB` access pattern that misses L1 on every iteration.

2. **Tiling reduces L1 misses dramatically.** CPU Counters show `mm_tiled` at $n = 1024$ sustains a much higher L1 hit rate than `mm_naive`, because the tile working set (96 KB at $T = 64$) fits within L1d (128 KB).

3. **MV kernels are bandwidth-bound.** Both row-major and col-major MV show near-zero L1 misses at small n and predictable streaming behaviour at large n. The profiler confirms that these kernels are limited by memory bandwidth, not compute.

8. Optimisation Strategy

Two optimisations were identified from compiler diagnostics and cache analysis.

8.1. Pointer Aliasing: `__restrict__`

The `-Rpass-missed=slp-vectorizer` output for kernels without `__restrict__` reported *"vectorization was possible but not beneficial"* and *"vectorization was impossible with available vectorization factors"* on every inner loop. The root cause: without aliasing information, the compiler must assume that writes through `result` could modify the memory pointed to by `matrixA` or `matrixB`, preventing it from reordering or widening loads into SIMD instructions.

Adding `__restrict__` to all pointer parameters constitutes a promise to the compiler that no two `restrict`-qualified pointers in the same scope alias. For matrix multiplication with a separate output buffer this invariant always holds. The effect is immediate: after the annotation, `-Rpass=loop-vectorize` confirms all five inner loops vectorise at width 2 with an interleave count of 4.

8.2. Tile Size Tuning

The tiled kernel's initial tile size was 32. On the target M4 Pro (128 KB L1d per P-core, 128-byte cache lines), three 32×32 tiles of `double` occupy $3 \times 32^2 \times 8 = 24$KB, only 19% of L1d. Increasing the tile to 64 raises the working set to $3 \times 64^2 \times 8 = 96$KB (75% of L1d), improving data reuse within the cache before eviction while leaving headroom for stack and loop state.

8.3. Combined Results

Table 8 shows the effect at $n = 1024$ under `-O2`. The tiled kernel with both optimisations achieves a 4.9× speedup over the naive baseline.

Variant	**Mean (ms)**	**Speedup**
`mm_naive`	937	1.0×
`mm_transposed_b`	494	1.9×
`mm_tiled` (tile=64)	193	4.9×

Table 8: Baseline vs optimised, $n = 1024$, `-O2`

The speedup compounds from two independent effects: transposing B fixes the column-stride access pattern (2×), and tiling adds L1 reuse on top of that (another 2.5×). Together with `__restrict__`-enabled vectorisation, the tiled kernel runs in under 200 ms at $n = 1024$ compared to nearly 1 second for naive.